

The Hash Function Cuishen

Submitters:

Yaobin Shen, Xiamen University

`yaobin.shen@xmu.edu.cn`

Tingting Cui, Hangzhou Dianzi University

`cuitingting@hdu.edu.cn`

Zhe Liu, Zhejiang University & Hangzhou Liang-An Technology Co., Ltd

`zhe.liu@zju.edu.cn`

Mingyang Chang, Xiamen University

`mingyang3326@stu.xmu.edu.cn`

Yan He, Hangzhou Dianzi University

`yan_he@hdu.edu.cn`

Feng Li, Xiamen University

`lifeng8425@stu.xmu.edu.cn`

Tianhao Liu, Zhejiang University & Hangzhou Liang-An Technology Co., Ltd

`lizyui@zju.edu.cn`

Corresponding submitter:

Tingting Cui, Hangzhou Dianzi University

`cuitingting@hdu.edu.cn`

June 30, 2026

Contents

1	Introduction	3
2	Notation	3
3	Hash Algorithm Cuishen	4
3.1	Message Padding	4
3.2	Position Counter Mechanism	5
3.3	Overall Structure	5
3.4	Compression Function	6
3.5	Finalization Function	7
4	Block Cipher Octarx	8
4.1	Overview	8
4.2	State	8
4.3	Design of the Cipher	9
4.4	Key Schedule	10
4.4.1	Message Expansion Part	10
4.4.2	Chaining-Value Update Part	11
4.4.3	Counter Update Part	12
4.5	Round Constants	13
5	Security Claims and Analyses	14
5.1	Generic Attacks	14
5.1.1	Indifferentiability	14
5.1.2	Collision Attacks	15
5.1.3	Preimage Attacks	15
5.1.4	Second Preimage Attacks	15
5.2	Dedicated Attacks	16
5.2.1	Differential Attacks	16
5.2.2	Meet-in-the-Middle Attacks	17
5.3	Other Security Properties	17
5.3.1	Length Extension Attacks (LEA)	17
5.3.2	Statistical Randomness Test Results	17
6	Performance	18
6.1	Software Performance	18
6.1.1	Performance analysis	18

6.1.2	Benchmark Setup	19
6.1.3	Performance Results and Resource Consumption	19
6.2	Hardware Implementation	22
7	Design Rationale	22
7.1	Properties of Overall Structure and Components	22
7.1.1	Overall Structure	22
7.1.2	Algorithm Component: the Octarx Block Cipher	23
7.2	Parameter Selection	24
8	Feature Declarations	25
8.1	Innovativeness	25
8.2	Simplicity	25
8.3	Flexibility and Extensibility	26
8.4	Implementation Performance Advantages	26

1 Introduction

As the cryptographic landscape transitions into the post-quantum era, the demand for hash functions that provide adequate security guarantees against both classical and quantum adversaries has become increasingly urgent. In response to the Call for New Generation Cryptographic Hash Algorithms by the Institute of Commercial Cryptography Standards (ICCS), we propose **Cuishen**, a high-performance and highly secure cryptographic hash algorithm. **Cuishen** supports digest sizes of 512, 768, and 1024 bits, accommodating messages up to $2^{128} - 1$ bits, thereby fulfilling modern cryptographic requirements with a balance of security and efficiency.

Cuishen employs a Double-Block-Length (DBL) Merkle-Damgård iterative hash mode built upon a custom-designed ARX (Addition, Rotation, XOR) block cipher called **Octarx**. To improve the algorithm against generic vulnerabilities, **Cuishen** integrates a 128-bit position counter into the underlying compression function. This position-dependent compression improves security against expandable message attacks and long-message second-preimage attacks. The traditional MD strengthening is replaced with a zero-padding mechanism, improving short-message hashing efficiency. Designed with simplicity and cross-platform adaptability in mind, the S-box-free **Octarx** core is easy to optimize. The overall structure of **Cuishen** allows it to act as a flexible cryptographic component, supporting an eXtendable-Output Function (XOF) mode alongside efficient MAC and KDF derivations.

The remainder of this document is organized as follows. Section 2 defines notation and Section 3 presents the complete algorithmic description of **Cuishen**, detailing the padding rules, underlying compression function, and operational mode. Section 4 specifies the underlying **Octarx** block cipher. Section 5 presents security claims and analyses. Section 6 presents performance evaluation. Finally, Section 7 and Section 8 elaborate on the design rationale and feature declarations.

2 Notation

Before describing the algorithm, we first define the notation, constants, and parameters used. The algorithm consistently employs big-endian format for data storage and computation.

Basic Parameters.

- n : The block size of the block cipher. According to the required output length, $n \in \{256, 512\}$.
- k : The key length of the block cipher.
- w : The word size. In this algorithm, $w = 64$.
- M : The message of hash input.
- ℓ : The bit length of the message M .
- t : The bit length of the hash output (e.g., 512, 768, 1024).
- B : The length of a single message block absorbed during iteration. In our mode, $B = 1024$.
- x : The number of '0's used for padding the message.
- Cnt : A 128-bit position counter recording the cumulative number of processed original message bits. Cnt_i is the counter at the i -th iteration.

Operators.

- $\oplus$: Bitwise exclusive OR (XOR) operation.
- $\boxplus$: Addition modulo 2^{64} .
- $\parallel$: Concatenation of bit strings.
- $\lll s$: Circular left shift by s bits.
- $[i]_n$: Represents the n -bit binary representation of the value i . For example, $[1]_{512}$ represents $0^{511}\parallel 1$.
- $\text{Truncate}(h, t)$: Truncating the digest h , retaining the leftmost (most significant) t bits.

3 Hash Algorithm Cuishen

This section describes the specification of **Cuishen** hash algorithm. The overall structure of **Cuishen** employs a MD iterative hashing mode based on a Double-Block-Length (DBL) structure and is inspired by Naito’s scheme [8]. We introduce a counter in the iteration of **Cuishen** to record the number of processed message bits and to improve security against second preimage attacks. In addition, we allow **Cuishen** to absorb one partial message block in the finalization function to improve efficiency. These changes further allow us to avoid using 10^* padding and length encoding to prevent redundant block. The underlying component of **Cuishen** is a dedicated block cipher called **Octarx** that will be introduced in the next section. In order to meet the requirements for outputs of different lengths as well as the corresponding security levels, the underlying block cipher **Octarx** is designed with block sizes 256, 512 bits, respectively.

3.1 Message Padding

As we will see later, the underlying compression function takes as input a counter that is the number of processed message bits. Therefore, the traditional length-appending padding mechanism (also known as MD strengthening) is no longer necessary. To avoid redundant compression function calls, **Cuishen** adopts a specific modulo-based zero padding mechanism. The message block length absorbed during each iteration is unified to $B = 1024$ bits for all instances of **Cuishen**. The concrete padding details depend on the size n of the block cipher.

ZeroPadding(M). Append x ‘0’ bits to the end of message M , where ℓ is the bit length of the message M , and x is the smallest non-negative integer satisfying:

- For $n = 256$: $\ell + x \equiv 768 \pmod{1024}$.
- For $n = 512$: $\ell + x \equiv 512 \pmod{1024}$.

The padded message M' is then parsed into two parts. The first part is divided into N iteration blocks of 1024 bits: $m_1, m_2, \dots, m_N$, where $N = \lfloor \ell/1024 \rfloor$. The second part is one residual block m_{final} , whose length is 768 bits for $n = 256$, and 512 bits for $n = 512$.

Remark. The N blocks of 1024 bits are absorbed during the normal iterations. The residual block m_{final} is reserved for the finalization function. If the original message length ℓ already satisfies the modulo condition, then $x = 0$, and no bits need to be padded.

3.2 Position Counter Mechanism

To improve the security against second preimage attacks, we introduce a 128-bit position counter Cnt . This counter is injected into the underlying compression function. In Cuishen, the 128-bit Cnt is used to recording the **cumulative number of processed original message bits**. The counter updating logic is defined as follows.

- **Empty Message** ($\ell = 0$): The iteration process is skipped. The initial state IV concatenated with the padded residual block m_{final} and the counter Cnt serves as the key for the finalization function. In this case, $Cnt = 0$.
- **Non-Empty Message** ($\ell > 0$):
 - *Iteration Phase*: For each 1024-bit message block absorbed, Cnt is updated to reflect the total number of original message bits processed up to that current block.
 - *Finalization Phase*: When the finalization function processes the residual block m_{final} (e.g., 768 bits for $n = 256$, or 512 bits for $n = 512$), the counter is updated based on the content of m_{final} . If m_{final} contains remaining message bits, Cnt is incremented by the remaining bit length. If m_{final} consists only of padded '0's (meaning no additional message bits are processed in this stage), Cnt remains unchanged from the last iteration.

3.3 Overall Structure

We describe each procedure in Cuishen. The pseudocode specification of Cuishen is presented in Algorithm 1, and a pictorial illustration of Cuishen is provided in Figure 1.

- **Initial Values (IV)**: The initial chaining variable $H_0 = t_0 \parallel b_0$ is initialized with pre-defined constants. These Initial Values are generated by taking the first 64 bits of the fractional parts of the square roots of consecutive Chen primes. To avoid overlap with the IV constants used in SHA-2, we select Chen primes greater than 100. Depending on the specific variant, the primes are assigned as follows.
 - Cuishen-512 ($n = 256$, requires 8 words): Uses the 1st to 8th Chen primes that are greater than 100: 101, 107, 109, 113, 127, 131, 137, 139.
 - Cuishen-768 ($n = 512$, requires 16 words): Uses the 9th to 24th Chen primes that are greater than 100: 149, 157, 167, 179, 181, 191, 197, 199, 211, 227, 233, 239, 251, 257, 263, 269.
 - Cuishen-1024 ($n = 512$, requires 16 words): Uses the 25th to 40th Chen primes that are greater than 100: 281, 293, 307, 311, 317, 337, 347, 353, 359, 379, 389, 401, 409, 419, 431, 443.
- **Iteration**: For each message block m_i ($1 \leq i \leq N$), the chaining variable is updated as follows:

$$H_i = t_i \parallel b_i \leftarrow \text{CF}_{\text{DBL}}(t_{i-1}, b_{i-1}, m_i, Cnt_i) \quad (1)$$

where CF_{DBL} is the compression function specified in Section 3.4.

- **Finalization**: $H_{N+1} = g_{\text{DBL}}(H_N, m_{final}, Cnt_{final})$, where g_{DBL} is the finalization function specified in Section 3.5.

Algorithm 1: The hash function Cuishen

```

1 Input: Message  $M$  of length  $\ell$  bits, target output length  $t$ 
2 Output: Hash value  $h$  of length  $t$  bits
   1: Initialization
   2:  $H_0 \leftarrow IV$  ▷ Initialize the  $2n$ -bit chaining variable ( $t_0 \parallel b_0$ )
   3:  $Cnt_0 \leftarrow 0$  ▷ Initialize the 128-bit position counter
   4: Padding and Parsing
   5:  $M' \leftarrow \text{ZeroPadding}(M)$  ▷ Append '0' bits according to modulo conditions
   6:  $N \leftarrow \lfloor \ell/1024 \rfloor$  ▷ Number of full 1024-bit message blocks
   7: Parse  $M'$  into  $N$  blocks of 1024 bits  $m_1, m_2, \dots, m_N$ , and a residual block  $m_{final}$ 
   8: Iteration Phase
   9: if  $\ell > 0$  then
  10:   for  $i \leftarrow 1$  to  $N$  do
  11:      $Cnt_i \leftarrow$  the cumulative number of original message bits up to  $m_i$ 
  12:     Parse  $H_{i-1}$  as  $t_{i-1} \parallel b_{i-1}$ 
  13:      $H_i \leftarrow \text{CF}_{\text{DBL}}(t_{i-1}, b_{i-1}, m_i, Cnt_i)$ 
  14:   end for
  15: end if
  16: Finalization Phase
  17:  $Cnt_{final} \leftarrow Cnt_N +$  remaining original message bits in  $m_{final}$ 
  18:  $Out_1 \parallel Out_2 \leftarrow g_{\text{DBL}}(H_N, m_{final}, Cnt_{final})$ 
  19: Output Generation
  20:  $h \leftarrow Out_1 \parallel Out_2$  ▷ For  $t \in \{512, 1024\}$ 
  21: if  $t = 768$  then
  22:    $h \leftarrow \text{Truncate}(Out_1 \parallel Out_2, 768)$  ▷ Retain the leftmost 768 bits
  23: end if
  24: return  $h$ 

```

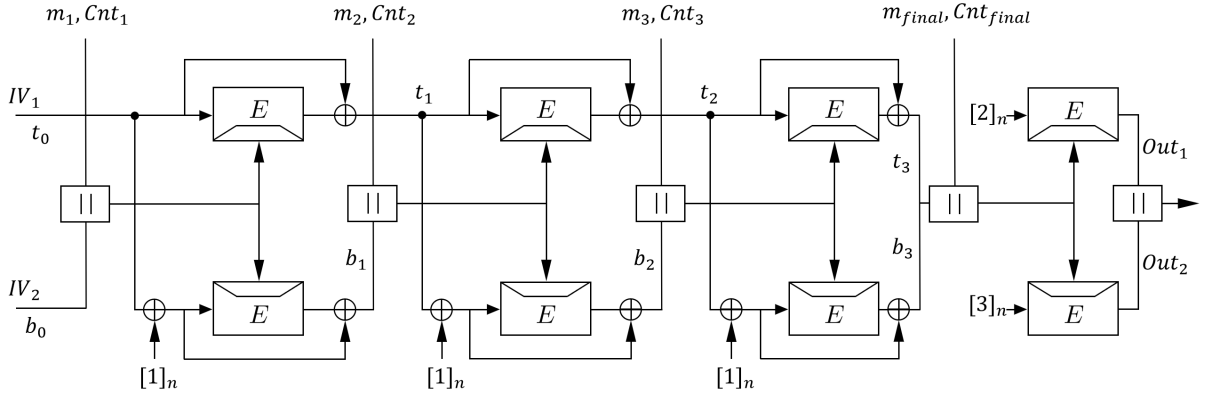


Figure 1: The overall structure of Cuishen mode.

3.4 Compression Function

The compression function $\text{CF}_{\text{DBL}} : \{0, 1\}^{2n} \times \{0, 1\}^{1024} \times \{0, 1\}^{128} \rightarrow \{0, 1\}^{2n}$ is constructed based on the double-block scheme by Naito [8], and includes an additional 128-bit counter to improve security against second preimage attacks. Here n is 256 for Cuishen-512, and 512 for Cuishen-768 and Cuishen-1024.

The compression function is illustrated in Figure 2, and is formally defined as $t_i \parallel b_i =$

$\text{CF}_{\text{DBL}}(t_{i-1}, b_{i-1}, m_i, \text{Cnt}_i)$, where

$$t_i = E(K_i, P_1) \oplus P_1 \quad (2)$$

$$b_i = E(K_i, P_2) \oplus P_2. \quad (3)$$

Here

- $E(K, P)$ is the underlying block cipher, taking a k -bit key K and a n -bit plaintext P as input, and outputting a n -bit ciphertext. Here $n = 256, k = 1408$ for Cuishen-512, and $n = 512, k = 1664$ for Cuishen-768 and Cuishen-1024.
- Key $K_i = m_i \parallel b_{i-1} \parallel \text{Cnt}_i$. The current message block m_i and the second half of the previous chaining variable b_{i-1} as well as position counter Cnt_i are concatenated as the key for the block cipher.
- $P_1 = t_{i-1}, P_2 = t_{i-1} \oplus [1]_n$.

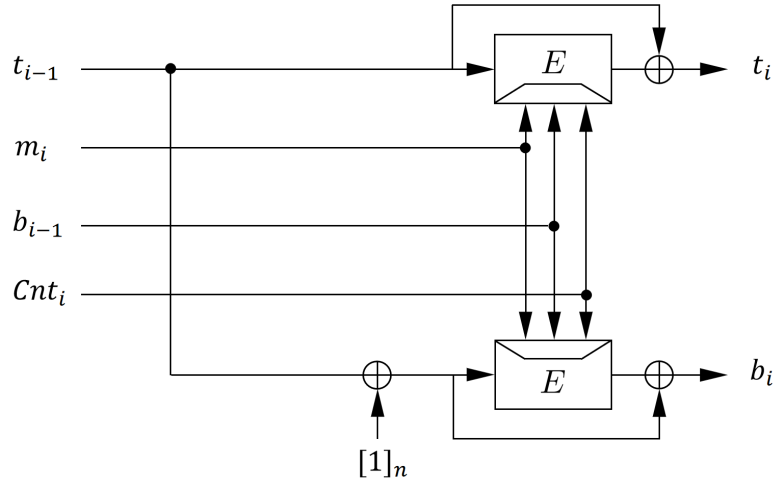


Figure 2: The compression function CF_{DBL} in Cuishen.

3.5 Finalization Function

The finalization function g_{DBL} is used to generate the final hash output. Formally, it is defined as $H_{N+1} = \text{Out}_1 \parallel \text{Out}_2 = g_{\text{DBL}}(t_N \parallel b_N, m_{\text{final}}, \text{Cnt}_{\text{final}})$, where

$$\text{Out}_1 = E(H_N \parallel m_{\text{final}} \parallel \text{Cnt}_{\text{final}}, [2]_n) \quad (4)$$

$$\text{Out}_2 = E(H_N \parallel m_{\text{final}} \parallel \text{Cnt}_{\text{final}}, [3]_n). \quad (5)$$

Here

- $H_N = t_N \parallel b_N$ is the output of the final iteration.
- The padded residual block m_{final} .
- The final position counter $\text{Cnt}_{\text{final}}$ that is updated according to the remaining message bits in m_{final} as defined in Section 3.2.
- Constant $[2]_n$ is $0^{n-2} \parallel 10$, and constant $[3]_n$ is $0^{n-2} \parallel 11$.

Output Generation: The algorithm defines the output based on the target length t .

- For Cuishen-512 and Cuishen-1024, the concatenated $2n$ -bit result $Out_1 \parallel Out_2$ exactly matches the target output length t . No truncation is needed.
- For Cuishen-768, the algorithm is instantiated with the 512-bit block cipher, which produces a 1024-bit final state. The final 768-bit hash value is obtained by truncating the 1024-bit result $Out_1 \parallel Out_2$, retaining only the leftmost (most significant) 768 bits.

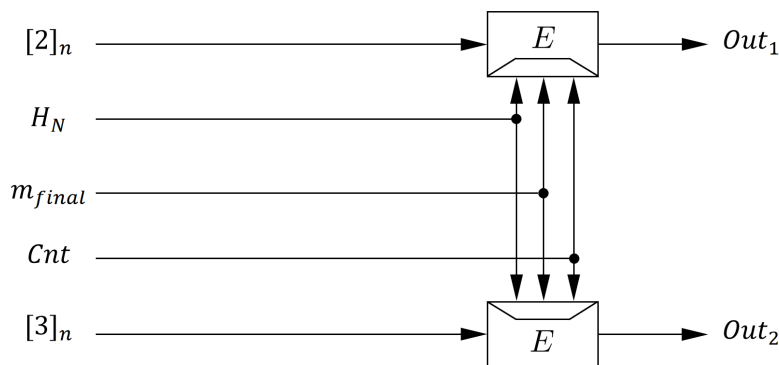


Figure 3: The finalization function g_{DBL} in Cuishen.

4 Block Cipher Octarx

In this section, we give the specification of **Octarx**, that is used to instantiate the underlying compression function of **Cuishen**. There are two versions of **Octarx**, namely **Octarx-256** with a 256-bit block size and **Octarx-512** with a 512-bit block size. **Octarx-256** can be used to instantiate Cuishen-512, while **Octarx-512** can be used to instantiate Cuishen-1024 and Cuishen-768. We remark that **Octarx** is specifically designed for the hash algorithm **Cuishen** and should not be used as a standalone block cipher.

4.1 Overview

Octarx is a word-oriented ARX block cipher family. Its round function is built from XOR, addition modulo 2^{64} , and circular left shifts on 64-bit words. In the present, two parameter sets are instantiated, namely **Octarx-256** and **Octarx-512**. Their main parameters are summarized in Table 1.

Table 1: Parameters of the block cipher algorithm **Octarx**.

Version	Block Length (bit)	Key Length (bit)	Total Rounds
Octarx-256	256	1408	64
Octarx-512	512	1664	64

4.2 State

Both versions of **Octarx** operate on 64-bit words, and all state words and key words are parsed in big-endian order. In each round, the master key is written uniformly as $K = M \parallel B \parallel Cnt$,

where M denotes the message part, B denotes the chaining-value part, and Cnt denotes the counter part.

For Octarx-256, the round- i input state is denoted by $X_i = (A_i, B_i, C_i, D_i)$. The corresponding key components are $M = (m_0, m_1, \dots, m_{15})$, $B = (b_0, b_1, b_2, b_3)$, and $Cnt = (c_0, c_1)$.

For Octarx-512, the input state is denoted by $X_i = (A_i, B_i, C_i, D_i, E_i, F_i, G_i, H_i)$. The corresponding key components are $M = (m_0, m_1, \dots, m_{15})$, $B = (b_0, b_1, \dots, b_7)$, and $Cnt = (c_0, c_1)$.

4.3 Design of the Cipher

The round function of Octarx adopts an ARX structure that combines word permutation with nonlinear word updates in each round.

For Octarx-256, the round key used in round i is denoted by $W_i = (w_i^a, w_i^b, w_i^c, w_i^d)$, the input state is $X_i = (A_i, B_i, C_i, D_i)$, and the round constant is denoted by $\mathbf{c}_i$. The round structure is illustrated in Figure 4.

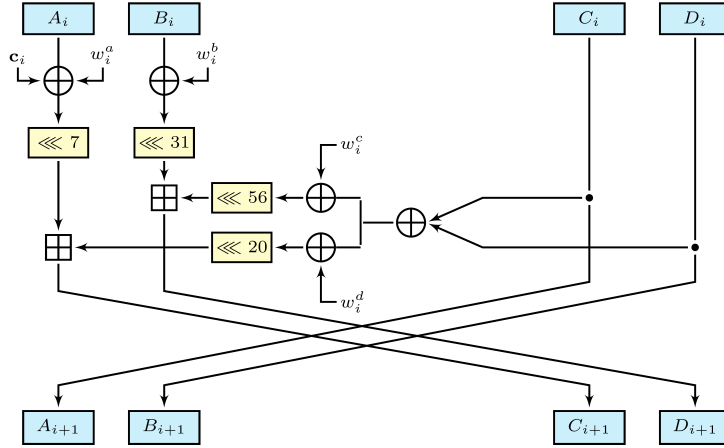


Figure 4: Round function of Octarx-256

For clarity, the round update of Octarx-256 can also be written in the pseudocode form shown in Algorithm 2.

Algorithm 2: Encryption procedure of Octarx-256

- 1: **for** $i \leftarrow 0$ **to** 63 **do**
 - 2: $T_i \leftarrow C_i \oplus D_i$
 - 3: $A_{i+1} \leftarrow C_i$
 - 4: $B_{i+1} \leftarrow D_i$
 - 5: $C_{i+1} \leftarrow ((A_i \oplus w_i^a \oplus \mathbf{c}_i) \lll 7) \boxplus ((T_i \oplus w_i^d) \lll 20)$
 - 6: $D_{i+1} \leftarrow ((B_i \oplus w_i^b) \lll 31) \boxplus ((T_i \oplus w_i^c) \lll 56)$
 - 7: **end for**
-

For Octarx-512, the round key is denoted by $W_i = (w_i^a, w_i^b, w_i^c, w_i^d, w_i^e, w_i^f, w_i^g, w_i^h)$, and the input state is $X_i = (A_i, B_i, C_i, D_i, E_i, F_i, G_i, H_i)$. The round constant is denoted by $\mathbf{c}_i$, and the round structure is illustrated in Figure 5.

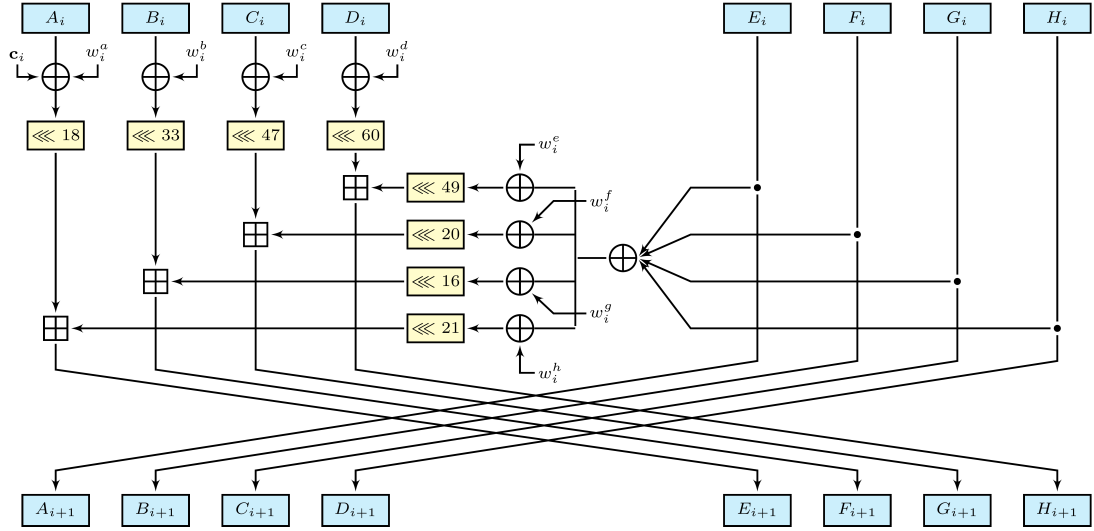


Figure 5: Round function of Octarx-512

For clarity, the encryption procedure of Octarx-512 can also be written in the pseudocode form shown in Algorithm 3.

Algorithm 3: Encryption procedure of Octarx-512

```

1: for  $i \leftarrow 0$  to 63 do
2:    $T_i \leftarrow E_i \oplus F_i \oplus G_i \oplus H_i$ 
3:    $A_{i+1} \leftarrow E_i$ 
4:    $B_{i+1} \leftarrow F_i$ 
5:    $C_{i+1} \leftarrow G_i$ 
6:    $D_{i+1} \leftarrow H_i$ 
7:    $E_{i+1} \leftarrow ((A_i \oplus w_i^a \oplus c_i) \lll 18) \boxplus ((T_i \oplus w_i^h) \lll 21)$ 
8:    $F_{i+1} \leftarrow ((B_i \oplus w_i^b) \lll 33) \boxplus ((T_i \oplus w_i^g) \lll 16)$ 
9:    $G_{i+1} \leftarrow ((C_i \oplus w_i^c) \lll 47) \boxplus ((T_i \oplus w_i^f) \lll 20)$ 
10:   $H_{i+1} \leftarrow ((D_i \oplus w_i^d) \lll 60) \boxplus ((T_i \oplus w_i^e) \lll 49)$ 
11: end for

```

4.4 Key Schedule

The key schedule is driven by three round-dependent parts: the 16-word message part M_i , the chaining-value part B_i , and the 2-word counter part Cnt_i . In round i , they are first used to derive the round key W_i , and are then updated for the next round.

4.4.1 Message Expansion Part

The message part evolves identically in both parameter sets. We write $M_i = (m_i^0, m_i^1, \dots, m_i^{15})$, and at round i the message-derived key words are selected as $k_i^{m_a} = m_i^0$, $k_i^{m_b} = m_i^1$, $k_i^{m_c} = m_i^8$, and $k_i^{m_d} = m_i^9$. The expansion pattern of the message part is shown in Figure 6.

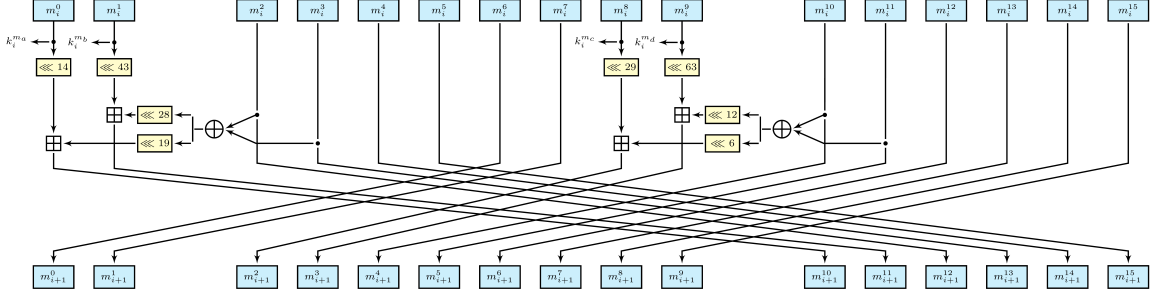


Figure 6: Expansion rule of the 1024-bit message register

4.4.2 Chaining-Value Update Part

For the chaining-value update part, Octarx-256 uses $B_i = (b_i^0, b_i^1, b_i^2, b_i^3)$, and at round i the chaining-value-derived key words are $k_i^{b_a} = b_i^0$, $k_i^{b_b} = b_i^1$, $k_i^{b_c} = b_i^2$, and $k_i^{b_d} = b_i^3$.

Its update pattern is shown in Figure 7.

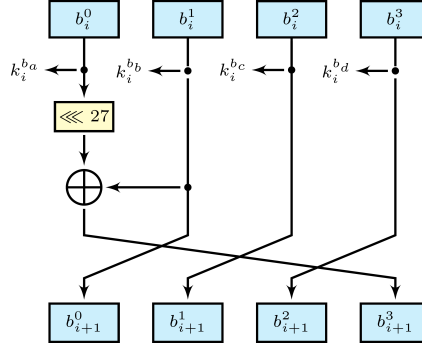


Figure 7: Update rule of the 256-bit chaining-value register

For Octarx-512, the chaining-value part is written as $B_i = (b_i^0, b_i^1, \dots, b_i^7)$, and at round i the chaining-value-derived key words are $k_i^{b_a} = b_i^0$, $k_i^{b_b} = b_i^1$, $k_i^{b_c} = b_i^2$, $k_i^{b_d} = b_i^3$, $k_i^{b_e} = b_i^4$, $k_i^{b_f} = b_i^5$, $k_i^{b_g} = b_i^6$, and $k_i^{b_h} = b_i^7$.

Its update pattern is shown in Figure 8.

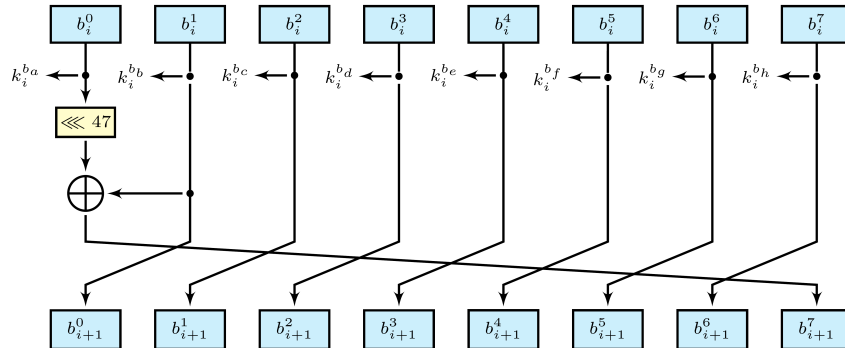


Figure 8: Update rule of the 512-bit chaining-value register

4.4.3 Counter Update Part

For both versions, the counter part is written as $Cnt_i = (c_i^0, c_i^1)$, and at round i the counter-derived key words are $k_i^{c_a} = c_i^0$ and $k_i^{c_b} = c_i^1$.

Its update pattern is shown in Figure 9.

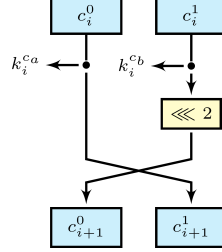


Figure 9: Update rule of the 128-bit counter register

The complete key schedule can also be described algorithmically. Since the two parameter sets differ in the update of the chaining-value part and in the extraction of round-key words, we present them separately in Algorithm 4 and Algorithm 5.

Algorithm 4: Key schedule of Octarx-256

```

1 Input:  $K = M_0 \| B_0 \| Cnt_0 = (m_0^0, m_0^1, \dots, m_0^{15}, b_0^0, b_0^1, \dots, b_0^3, c_0^0, c_0^1)$ 
2 Output:  $W_0, W_1, \dots, W_{63}$ 
3 for  $i \leftarrow 0$  to 63 do
4   // Process the message part
5    $k_i^{m_a} \leftarrow m_i^0, k_i^{m_b} \leftarrow m_i^1, k_i^{m_c} \leftarrow m_i^8, k_i^{m_d} \leftarrow m_i^9$ 
6    $m_{i+1}^0 \leftarrow m_i^6, m_{i+1}^1 \leftarrow m_i^7$ 
7    $m_{i+1}^2 \leftarrow (m_i^8 \lll 29) \oplus ((m_i^{10} \oplus m_i^{11}) \lll 6)$ 
8    $m_{i+1}^3 \leftarrow (m_i^9 \lll 63) \oplus ((m_i^{10} \oplus m_i^{11}) \lll 12)$ 
9    $m_{i+1}^4 \leftarrow m_i^{10}, m_{i+1}^5 \leftarrow m_i^{11}, m_{i+1}^6 \leftarrow m_i^{12}, m_{i+1}^7 \leftarrow m_i^{13}$ 
10   $m_{i+1}^8 \leftarrow m_i^{14}, m_{i+1}^9 \leftarrow m_i^{15}$ 
11   $m_{i+1}^{10} \leftarrow (m_i^0 \lll 14) \oplus ((m_i^2 \oplus m_i^3) \lll 19)$ 
12   $m_{i+1}^{11} \leftarrow (m_i^1 \lll 43) \oplus ((m_i^2 \oplus m_i^3) \lll 28)$ 
13   $m_{i+1}^{12} \leftarrow m_i^2, m_{i+1}^{13} \leftarrow m_i^3, m_{i+1}^{14} \leftarrow m_i^4, m_{i+1}^{15} \leftarrow m_i^5$ 
14  // Process the chaining-value part
15   $k_i^{b_a} \leftarrow b_i^0, k_i^{b_b} \leftarrow b_i^1, k_i^{b_c} \leftarrow b_i^2, k_i^{b_d} \leftarrow b_i^3$ 
16   $b_{i+1}^0 \leftarrow b_i^1, b_{i+1}^1 \leftarrow b_i^2, b_{i+1}^2 \leftarrow b_i^3, b_{i+1}^3 \leftarrow (b_i^0 \lll 27) \oplus b_i^1$ 
17  // Process the counter part
18   $k_i^{c_a} \leftarrow c_i^0, k_i^{c_b} \leftarrow c_i^1$ 
19   $c_{i+1}^0 \leftarrow c_i^1, c_{i+1}^1 \leftarrow c_i^0 \lll 2$ 
20  // Output the round key
21   $w_i^a \leftarrow k_i^{m_a} \oplus k_i^{b_a} \oplus k_i^{c_a}, w_i^b \leftarrow k_i^{m_b} \oplus k_i^{b_b} \oplus k_i^{c_b}$ 
22   $w_i^c \leftarrow k_i^{m_c} \oplus k_i^{b_c}, w_i^d \leftarrow k_i^{m_d} \oplus k_i^{b_d}$ 
23   $W_i \leftarrow (w_i^a, w_i^b, w_i^c, w_i^d)$ 
24 end for

```

Algorithm 5: Key schedule of Octarx-512

```
1 Input:  $K = M_0 \| B_0 \| Cnt_0 = (m_0^0, m_0^1, \dots, m_0^{15}, b_0^0, b_0^1, \dots, b_0^7, c_0^0, c_0^1)$ 
2 Output:  $W_0, W_1, \dots, W_{63}$ 
1: for  $i \leftarrow 0$  to 63 do
2:   // Process the message part
3:    $\mathbf{k}_i^{m_a} \leftarrow m_i^0, \mathbf{k}_i^{m_b} \leftarrow m_i^1, \mathbf{k}_i^{m_c} \leftarrow m_i^8, \mathbf{k}_i^{m_d} \leftarrow m_i^9$ 
4:    $m_{i+1}^0 \leftarrow m_i^6, m_{i+1}^1 \leftarrow m_i^7$ 
5:    $m_{i+1}^2 \leftarrow (m_i^8 \lll 29) \boxplus ((m_i^{10} \oplus m_i^{11}) \lll 6)$ 
6:    $m_{i+1}^3 \leftarrow (m_i^9 \lll 63) \boxplus ((m_i^{10} \oplus m_i^{11}) \lll 12)$ 
7:    $m_{i+1}^4 \leftarrow m_i^{10}, m_{i+1}^5 \leftarrow m_i^{11}, m_{i+1}^6 \leftarrow m_i^{12}, m_{i+1}^7 \leftarrow m_i^{13}$ 
8:    $m_{i+1}^8 \leftarrow m_i^{14}, m_{i+1}^9 \leftarrow m_i^{15}$ 
9:    $m_{i+1}^{10} \leftarrow (m_i^0 \lll 14) \boxplus ((m_i^2 \oplus m_i^3) \lll 19)$ 
10:   $m_{i+1}^{11} \leftarrow (m_i^1 \lll 43) \boxplus ((m_i^2 \oplus m_i^3) \lll 28)$ 
11:   $m_{i+1}^{12} \leftarrow m_i^2, m_{i+1}^{13} \leftarrow m_i^3, m_{i+1}^{14} \leftarrow m_i^4, m_{i+1}^{15} \leftarrow m_i^5$ 
12:  // Process the chaining-value part
13:   $\mathbf{k}_i^{b_a} \leftarrow b_i^0, \mathbf{k}_i^{b_b} \leftarrow b_i^1, \mathbf{k}_i^{b_c} \leftarrow b_i^2, \mathbf{k}_i^{b_d} \leftarrow b_i^3$ 
14:   $\mathbf{k}_i^{b_e} \leftarrow b_i^4, \mathbf{k}_i^{b_f} \leftarrow b_i^5, \mathbf{k}_i^{b_g} \leftarrow b_i^6, \mathbf{k}_i^{b_h} \leftarrow b_i^7$ 
15:   $b_{i+1}^0 \leftarrow b_i^1, b_{i+1}^1 \leftarrow b_i^2, b_{i+1}^2 \leftarrow b_i^3, b_{i+1}^3 \leftarrow b_i^4$ 
16:   $b_{i+1}^4 \leftarrow b_i^5, b_{i+1}^5 \leftarrow b_i^6, b_{i+1}^6 \leftarrow b_i^7, b_{i+1}^7 \leftarrow (b_i^0 \lll 47) \oplus b_i^1$ 
17:  // Process the counter part
18:   $\mathbf{k}_i^{c_a} \leftarrow c_i^0, \mathbf{k}_i^{c_b} \leftarrow c_i^1$ 
19:   $c_{i+1}^0 \leftarrow c_i^1, c_{i+1}^1 \leftarrow c_i^0 \lll 2$ 
20:  // Output the round key
21:   $w_i^a \leftarrow \mathbf{k}_i^{m_a} \oplus \mathbf{k}_i^{b_a} \oplus \mathbf{k}_i^{c_a}, w_i^b \leftarrow \mathbf{k}_i^{m_b} \oplus \mathbf{k}_i^{b_b} \oplus \mathbf{k}_i^{c_b}$ 
22:   $w_i^c \leftarrow \mathbf{k}_i^{m_c} \oplus \mathbf{k}_i^{b_c}, w_i^d \leftarrow \mathbf{k}_i^{m_d} \oplus \mathbf{k}_i^{b_d}$ 
23:   $w_i^e \leftarrow \mathbf{k}_i^{b_e}, w_i^f \leftarrow \mathbf{k}_i^{b_f}, w_i^g \leftarrow \mathbf{k}_i^{b_g}, w_i^h \leftarrow \mathbf{k}_i^{b_h}$ 
24:   $W_i \leftarrow (w_i^a, w_i^b, w_i^c, w_i^d, w_i^e, w_i^f, w_i^g, w_i^h)$ 
25: end for
```

4.5 Round Constants

For both Octarx-256 and Octarx-512, the round constants are denoted uniformly by $\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{63}$. Each $\mathbf{c}_i$ is a 64-bit word. For Octarx-256, the constants are derived from the hexadecimal digits of the fractional part of π :

0x243f6a8885a308d3	0x13198a2e03707344	0xa4093822299f31d0	0x082efa98ec4e6c89
0x452821e638d01377	0xbe5466cf34e90c6c	0xc0ac29b7c97c50dd	0x3f84d5b5b5470917
0x9216d5d98979fb1b	0xd1310ba698dfb5ac	0x2ffd72dbd01adfb7	0xb8e1afed6a267e96
0xba7c9045f12c7f99	0x24a19947b3916cf7	0x0801f2e2858efc16	0x636920d871574e69
0xa458fea3f4933d7e	0x0d95748f728eb658	0x718bcd5882154aee	0x7b54a41dc25a59b5
0x9c30d5392af26013	0xc5d1b023286085f0	0xca417918b8db38ef	0x8e79dcb0603a180e
0x6c9e0e8bb01e8a3e	0xd71577c1bd314b27	0x78af2fda55605c60	0xe65525f3aa55ab94
0x5748986263e81440	0x55ca396a2aab10b6	0xb4cc5c341141e8ce	0xa15486af7c72e993
0xb3ee1411636fbc2a	0x2ba9c55d741831f6	0xce5c3e169b87931e	0xafd6ba336c24cf5c
0x7a32538128958677	0x3b8f48986b4bb9af	0xc4bfe81b66282193	0x61d809ccfb21a991
0x487cac605dec8032	0xef845d5de98575b1	0xdc262302eb651b88	0x23893e81d396acc5
0x0f6d6ff383f44239	0x2e0b4482a4842004	0x69c8f04a9e1f9b5e	0x21c66842f6e96c9a
0x670c9c61abd388f0	0x6a51a0d2d8542f68	0x960fa728ab5133a3	0x6eef0b6c137a3be4
0xba3bf0507efb2a98	0xa1f1651d39af0176	0x66ca593e82430e88	0x8cee8619456f9fb4
0x7d84a5c33b8b5ebe	0xe06f75d885c12073	0x401a449f56c16aa6	0x4ed3aa62363f7706
0x1bfedf72429b023d	0x37d0d724d00a1248	0xdb0fead349f1c09b	0x075372c980991b7b

For Octarx-512, the constants are derived from the hexadecimal digits of the fractional part of the natural constant e :

0xb7e151628aed2a6a	0xbf7158809cf4f3c7	0x62e7160f38b4da56	0xa784d9045190cfef
0x324e7738926cfbe5	0xf4bf8d8d8c31d763	0xda06c80abb1185eb	0x4f7c7b5757f59584
0x90cfd47d7c19bb42	0x158d9554f7b46bce	0xd55c4d79fd5f24d6	0x613c31c3839a2ddf
0x8a9a276bcfbfa1c8	0x77c56284dab79cd4	0xc2b3293d20e9e5ea	0xf02ac60acc93ed87
0x4422a52ecb238fee	0xe5ab6add835fd1a0	0x753d0a8f78e537d2	0xb95bb79d8dcaec64
0x2c1e9f23b829b5c2	0x780bf38737df8bb3	0x00d01334a0d0bd86	0x45cbfa73a6160ffe
0x393c48cbbca060f	0x0ff8ec6d31beb5cc	0xeed7f2f0bb088017	0x163bc60df45a0ecb
0x1bcd289b06cbbfea	0x21ad08e1847f3f73	0x78d56ced94640d6e	0xf0d3d37be67008e1
0x86d1bf275b9b241d	0xeb64749a47dfdfb9	0x6632c3eb061b6472	0xbbf84c26144e49c2
0xd04c324ef10de513	0xd3f5114b8b5d374d	0x93cb8879c7d52ffd	0x72ba0aae7277da7b
0xa1b4af1488d8e836	0xaf14865e6c37ab68	0x76fe690b57112138	0x2af341afe94f77bc
0xf06c83b8ff5675f0	0x979074ad9a787bc5	0xb9bd4b0c5937d3ed	0xe4c3a79396215eda
0xb1f57d0b5a7db461	0xdd8f3c75540d0012	0x1fd56e95f8c731e9	0xc4d7221bbcd0c62b
0xb5a87804b679a0ca	0xa41d802a4604c311	0xb71de3e5c6b400e0	0x24a6668ccf2e2de8
0x6876e4f5c50000f0	0xa93b3aa7e6342b30	0x2a0a47373b25f73e	0x3b26d569fe2291ad
0x36d6a147d1060b87	0x1a2801f978376408	0x2ff592d9140db1e9	0x399df4b0e14ca8e8

In both versions, $\mathbf{c}_i$ is injected only into the branch associated with A_i .

5 Security Claims and Analyses

This section gives the security claims and analyses of both Cuishen hash algorithm and its underlying block cipher Octarx. Table 2 presents the claimed security strengths for Cuishen against generic attacks.

Table 2: Security strength claims for Cuishen DBL mode.

Output t	Block n	Collision	Preimage	Sec Preimage
512	256	2^{256}	2^{512}	2^{512}
768	512	2^{384}	2^{768}	2^{768}
1024	512	2^{512}	2^{1024}	2^{1024}

5.1 Generic Attacks

5.1.1 Indifferentiability

The overall indifferentiability of this construction is closely related to the indifferentiability proof established by Naito [8]. In Naito’s construction, indifferentiability is achieved through the Merkle-Damgård iterative structure together with a finalization function that encrypts fixed constants using the final internal state as the key. Cuishen adopts the same iterative structure and a similar finalization mechanism. Although Cuishen introduces an additional counter in each iteration and absorbs one partial message block during the finalization phase, the additional counter can only improve the security and the partial message block has no influence on the security bound as far as we analyzed. Hence, we expect these modifications to have only a limited impact on the indifferentiability analysis and Cuishen can achieve the same $O(2^n)$ indifferentiability as Naito’s scheme [8].

5.1.2 Collision Attacks

We evaluate the collision resistance of Cuishen under both classical and quantum computing conditions.

- **Classical Collision Resistance.** According to the birthday paradox, the generic time complexity of finding a collision for a hash value of length t is $O(2^{t/2})$. Since Cuishen adopts the double-block-length mode [6, 8], it implies that the classical collision resistance of Cuishen can reach 2^n . Hence, Cuishen-512 and Cuishen-1024 provide collision resistance up to bounds of 2^{256} and 2^{512} , respectively. For Cuishen-768, the algorithm employs the $n = 512$ version with an internal state of 1024 bits, truncating the final hash output to $t = 768$ bits. This acts as a wide-pipe design. Hence, it also provides collision resistance up to bound of 2^{384} .
- **Quantum Collision Resistance:** For the quantum collision problem, the BHT algorithm provides a generic theoretical lower bound of $O(2^{t/3})$ [3]. Therefore, the instances ($t = 512, 768, 1024$) achieve a theoretical quantum collision resistance of 170, 256, 341 bits, respectively. Although the BHT algorithm theoretically achieves $O(2^{t/3})$, its implementation requires $O(2^{t/3})$ quantum memory. Considering the physical overhead constraints of limited qRAM, the actual cost of a quantum parallel collision search far exceeds theoretical steps [4]. The quantum collision bounds claimed above are conservative.

5.1.3 Preimage Attacks

We evaluate the preimage resistance of Cuishen under both classical and quantum computing conditions.

- **Classical Preimage Resistance:** According to exhaustive search, the generic complexity of finding a preimage is $O(2^t)$. The design of the finalization function break the invertibility of the overall structure. Given a hash value $h = Out_1 \parallel Out_2$, finding a preimage is equivalent to find a key K such that $E^{-1}(K, Out_1) = [2]_n$ and $E^{-1}(K, Out_2) = [3]_n$. This guarantees that the classical preimage resistance achieves the bound $O(2^{2n})$ (i.e., 2^{512} , 2^{768} and 2^{1024} for Cuishen-512, Cuishen-768, and Cuishen-1024, respectively).
- **Quantum Preimage Resistance:** Due to the finalization function, reversing the hash value is transformed into a key-searching problem. Attackers can rely on Grover's algorithm [5], which reduces the time complexity via quantum amplitude amplification. Therefore, the quantum preimage resistance remains at $O(2^{t/2})$, e.g., offering 256 bits, 384 bits, and 512 bits of quantum preimage resistance for Cuishen-512, Cuishen-768, and Cuishen-1024, respectively.

5.1.4 Second Preimage Attacks

We evaluate the second preimage resistance of Cuishen under both classical and quantum computing conditions.

- **Classical and Quantum Second Preimage Resistance:** Following similar rationale as preimage resistance, the design of the finalization function guarantees that classical second preimage resistance achieves the bound $O(2^{2n})$ (i.e., 2^{512} , 2^{768} , and 2^{1024} for Cuishen-512, Cuishen-768, and Cuishen-1024, respectively), and quantum second preimage resistance remains at $O(2^{t/2})$ (i.e., 2^{256} , 2^{384} , and 2^{512} for Cuishen-512, Cuishen-768,

and Cuishen-1024, respectively). Furthermore, because a counter is injected into each iteration, generic long-message second-preimage attacks are resisted [7].

5.2 Dedicated Attacks

To evaluate the ability of Cuishen against collision attack, second preimage attack and preimage attack, the security of Octarx should be carefully considered as well. We give some evaluated results on differential attacks and meet-in-the-middle attacks.

5.2.1 Differential Attacks

A collision in Cuishen occurs when two different messages produce the same hash value, which is analogous to related-key and known-key differential attacks on block cipher Octarx. Thus, we consider four cases in Cuishen as follows:

- **Case 1: assume $\Delta t_{i-1} = 0$, $\Delta b_{i-1} = 0$, $\Delta t_i = 0$, the attacker can control one-block message and the lsb 10-bit ΔCnt_i .** This is the most direct approach to finding a collision. The attacker controls two messages such that the inputs to the block cipher are identical, and certain key bits (i.e., chaining-value b_{i-1}) coincide. By utilizing distinct new message blocks to force identical outputs, a collision can be successfully constructed. This scenario is analogous to related-key differential cryptanalysis in block ciphers, but both the input and output differences are zero. Under this framework, we employed an automated search methodology. The results indicate that for the Octarx-256 variant, the differential probability are 2^{-2} for 4 rounds, 2^{-5} for 5 rounds and 2^{-13} for 6 rounds. For the Octarx-512 variant, the differential probability is 2^{-6} for 4 rounds and 2^{-16} for 5 rounds.
- **Case 2: assume $\Delta t_{i-1} = 0$, $\Delta b_{i-1} = 0$, the attacker can control one-block message and the lsb 10-bit ΔCnt_i to produce any expected Δt_i .** Compared to Case 1, Case 2 relaxes the constraints on the output difference. This adjustment is motivated by collision attack scenarios where the adversary typically utilizes two message blocks to construct a collision. Specifically, the first message block induces a transition from a zero input difference to a non-zero output difference in the block cipher (which serves as the input difference for the subsequent block cipher invocation), while the second message block facilitates a transition from a non-zero difference to a zero output difference, thereby successfully achieving a collision. Under this scenario, we employed an automated search methodology. For the Octarx-256 variant, the results indicate that the differential probabilities are 2^{-1} for 4 rounds, 2^{-5} for 5 rounds and 2^{-13} for 6 rounds. Similarly, for the Octarx-512 variant, the differential probabilities are 2^{-1} for 4 rounds, 2^{-8} for 5 rounds and no more than 2^{-14} for 6 rounds.
- **Case 3: assume the input difference of Octarx is $\Delta in = 2 \oplus 3 = 1$, the attacker can control one-block message and Cnt_i to produce the expected output difference of Octarx.** In the final compression function of Cuishen, the Octarx block cipher is invoked twice with inputs of 2 and 3, respectively, under a shared key. Investigating whether the two outputs of Octarx exhibit strong correlation is of significant importance. This problem is analogous to studying whether there exist high-probability single-key differential trails in the Octarx block cipher when the input difference is 1. Under this scenario, the results indicate that for the Octarx-256 variant, the differential probabilities are 2^{-41} for 5 rounds, 2^{-64} for 6 rounds and 2^{-80} for 7 rounds. Similarly, for the Octarx-512 variant,

the differential probabilities drop to 2^{-40} for 3 rounds, 2^{-78} for 4 rounds and 2^{-108} for 5 rounds.

- **Case 4: assume Δt_{i-1} , Δb_{i-1} and Δt_i are all free, the attacker can control one-block message and Cnt_i .** Although we consider this hypothetical scenario unlikely to be directly applicable to practical attacks against the Cuishen hash function, we have nonetheless conducted an analysis to provide a more comprehensive security evaluation. The results indicate that for the Octarx-256 variant, the related-key differential probabilities are 2^{-2} for 5 rounds, 2^{-6} for 6 rounds and no more than 2^{-10} for 7 rounds. Similarly, for the Octarx-512 variant, the related-key differential probabilities are 2^{-2} for 5 rounds, 2^{-10} for 6 rounds and no more than 2^{-19} for 7 rounds.

The search complexity for related-key differential trails increases exponentially with the number of rounds, making it difficult to establish a definitive bound for the exact number of secure rounds. However, the rapid decay in differential probability suggests that finding expected key (i.e., that messages in Cuishen) within a time complexity of 2^{256} for Octarx-256 variant or 2^{512} for Octarx-512 variant is highly improbable.

5.2.2 Meet-in-the-Middle Attacks

The meet-in-the-middle (MITM) attack is a crucial low-data-complexity cryptanalytic technique for key recovery in block ciphers, as well as a prominent method for preimage attacks on hash functions. Since the Cuishen hash function is constructed upon a block cipher primitive, it is essential to evaluate its resistance against MITM attacks.

For the Octarx-256 variant, full diffusion is achieved after 5 rounds, where each output bit is influenced by all input bits. After 7 rounds, every output bit depends on the entire 1024-bit key (i.e., the message). Furthermore, after 5 rounds, each output bit is affected by the 256-bit key (i.e., the chaining-value b_{i-1}) and the 128-bit key (i.e., the counter Cnt_i), respectively. Consequently, based on the meet-in-the-middle principle, the number of attackable rounds is bounded by 14.

For the Octarx-512 variant, full diffusion is achieved after 4 rounds, where each output bit is influenced by all input bits. After 7 rounds, every output bit depends on the entire 1024-bit key (i.e., the message). Furthermore, after 4 rounds, each output bit is affected by both the 256-bit key (i.e., the chaining-value b_{i-1}) and the 128-bit key (i.e., the counter Cnt_i). Consequently, based on the meet-in-the-middle principle, the number of attackable rounds is bounded by 14.

5.3 Other Security Properties

5.3.1 Length Extension Attacks (LEA)

Traditional Merkle-Damgård constructs are vulnerable to LEA. In Cuishen, the finalization function encrypts fixed constants using the final iteration state combined with the position counter and the residual message block. This finalization function prevents an attacker from using the final hash output as a valid chaining value to append arbitrary data.

5.3.2 Statistical Randomness Test Results

We evaluate the statistical randomness of our algorithm using the NIST SP 800-22 statistical test suite. Following the recommended practice, we generate 1,000 sequences of length 10^6

bits for each of the three Cuishen variants (512, 768, and 1024). For each test, a sequence is considered to have passed if its P-value is at least 0.01, and the test is considered passed if the pass rate across all sequences is at least 98%.

Table 3 summarises the test results. Pass rates are given as X/Y, where X is the number of passing sequences and Y is the total number evaluated for that test. For most tests, Y = 1000; for RandomExcursions and RandomExcursionsVariant, Y is the number of sequences that exhibit at least one zero-crossing in the cumulative sum random walk—sequences that never return to zero are excluded, and the corresponding Y values are listed in the table. All 15 tests are passed for every Cuishen variant, demonstrating that the outputs of the Cuishen hash function exhibit statistical randomness and meet the requirements of the NIST SP 800-22 suite.

Table 3: Summary of NIST SP 800-22 test results for Cuishen.

Statistical Test	Cuishen-512	Cuishen-768	Cuishen-1024
Frequency	993/1000	988/1000	984/1000
BlockFrequency	988/1000	993/1000	991/1000
CumulativeSums	993/1000	990/1000	985/1000
Runs	985/1000	991/1000	989/1000
LongestRun	991/1000	994/1000	983/1000
Rank	990/1000	992/1000	993/1000
FFT	982/1000	988/1000	985/1000
NonOverlappingTemplate	982/1000	980/1000	984/1000
OverlappingTemplate	992/1000	987/1000	983/1000
Universal	986/1000	993/1000	989/1000
ApproximateEntropy	991/1000	987/1000	986/1000
RandomExcursions	607/617	615/624	592/601
RandomExcursionsVariant	610/617	617/624	593/601
Serial	988/1000	993/1000	992/1000
LinearComplexity	991/1000	990/1000	990/1000

6 Performance

6.1 Software Performance

6.1.1 Performance analysis

Cuishen’s software performance is mainly determined by its fixed 1024-bit block-compression path. The algorithm follows a linear chaining design: each message block consumes the chaining state produced by the previous block and generates the new state required by the next block. Therefore, for a single long message, the execution naturally proceeds with inter-block dependencies. The main focus of performance analysis is not block-level parallel compression, but the efficiency of the Cuishen round function and the key schedule within each block.

This design also provides a clear and favorable performance profile. Cuishen’s core operations are 64-bit XOR, modular addition, and word rotation, which map directly to the native datapath of mainstream 64-bit processors. Although consecutive message blocks are processed in sequence, the computation inside each round contains multiple word-level state updates, two parallel encryption branches, and round-key-related arithmetic that can be kept in registers and executed with instruction-level or vector-lane parallelism. In addition, Cuishen’s XOR-

and rotation-oriented operation patterns can fit ARMv8.2-crypto extension instructions well, so ARM implementations can reuse instructions such as EOR3 and XAR to provide additional architecture-specific optimization space. As a result, long-message throughput mainly reflects the efficiency of the steady-state block-compression loop, while empty and short-message results are more affected by call overhead, tail handling, and the fixed finalization step.

6.1.2 Benchmark Setup

This subsection reports the software self-evaluation results in the format required by the NGCC x86 and ARM implementation self-evaluation guides. The items reported are algorithm and implementation information, test environment, function correctness, S1–S8 hash performance, resource consumption, transfer/storage overhead, and evidence index. Only self-evaluation records are listed in this section.

For hash algorithms, the NGCC guides define S1–S8 as fixed input sizes from 32 bytes to 65536 bytes. For each case, the benchmark records the average running time, average cycle count computed from the platform frequency, and byte-processing throughput in MB/s. Each case was executed at least 1000 times and was extended to at least one second of measurement time. The input messages were generated by a fixed LCG with seed `0x4e474343`; KAT generation used the official DRNG and KAT harness supplied for hash submissions.

The evaluation environment and build configuration are summarized below.

Table 4: Evaluation environment and build configuration.

Platform	Processor and memory	OS / kernel	Compiler / build tool	Main compile flags
ARMv8	HiSilicon Kunpeng 920 2c4g single core: CPU 0	Ubuntu 24.04 Linux 6.8.0	GCC 13.3.0 direct GCC build	-O3 -std=c99 -march=armv8.2-a+sha3 -Wpedantic -Wall -Wextra
Intel Xeon Server AVX2/BMI2	Intel Xeon Platinum 8383C @ 2.70GHz 160 CPU, 2 socket single core: CPU 0	Ubuntu 22.04 Linux 5.15.0	GCC 11.4.0 direct GCC build	-O3 -std=c99 -mavx2 -mbmi -mbmi2 -madx -mtune=native
Intel Xeon Server AVX512	Intel Xeon Platinum 8383C @ 2.70GHz 160 CPU, 2 socket single core: CPU 0	Ubuntu 22.04 Linux 5.15.0	GCC 11.4.0 direct GCC build	-O3 -std=c99 -mavx2 -mavx512f -mavx512vl -mtune=native -fno-stack-protector

The average-cycle column below is computed from the measured average time and the platform frequency for the ARM platform, using 2.0 GHz for the Huawei Cloud instance. For the x86 results, the column reports the `rdtsc_ticks` recorded on the Intel Xeon Server during measurement.

6.1.3 Performance Results and Resource Consumption

The NGCC S1–S8 software performance results for the ARMv8 optimized implementation, the x86 AVX2/BMI2 assembly implementation, and the x86 AVX-512 C implementation are reported in Tables 5, 6, and 7, respectively. The corresponding resource-consumption results are summarized in Table 8.

Table 5: ARMv8 optimized implementation, NGCC S1–S8 performance results.

Version	Level	Message Length (Bytes)	Execution Time (ns)	Clock Cycles (cycles)	Throughput (MB/s)
Cuishen-512	S1	32	200.07	400.14	159.94
	S2	128	389.54	779.08	328.59
	S3	512	974.94	1949.88	525.16
	S4	1024	1755.57	3511.14	583.29
	S5	4096	6523.86	13047.72	627.85
	S6	8192	12769.47	25538.94	641.53
	S7	16384	25326.33	50652.66	646.92
	S8	65536	100149.66	200299.32	654.38
Cuishen-768	S1	32	347.96	695.92	91.97
	S2	128	669.19	1338.38	191.28
	S3	512	1676.70	3353.40	305.36
	S4	1024	3022.19	6044.38	338.83
	S5	4096	11083.04	22166.08	369.57
	S6	8192	21830.75	43661.50	375.25
	S7	16384	43367.88	86735.76	377.79
	S8	65536	172332.46	344664.92	380.29
Cuishen-1024	S1	32	345.46	690.92	92.63
	S2	128	671.65	1343.30	190.58
	S3	512	1677.13	3354.26	305.28
	S4	1024	3019.23	6038.46	339.16
	S5	4096	11075.00	22150.00	369.84
	S6	8192	21779.95	43559.90	376.13
	S7	16384	43293.52	86587.04	378.44
	S8	65536	171920.23	343840.46	381.20

Table 6: x86 AVX2/BMI2 assembly implementation, NGCC S1–S8 performance results.

Version	Level	Message Length (Bytes)	Execution Time (ns)	Clock Cycles (cycles)	Throughput (MB/s)
Cuishen-512	S1	32	246.57	664.23	129.78
	S2	128	446.66	1203.25	286.57
	S3	512	1042.62	2808.71	491.07
	S4	1024	1838.08	4951.59	557.10
	S5	4096	6619.70	17832.78	618.76
	S6	8192	12989.03	34991.08	630.69
	S7	16384	25716.64	69277.92	637.10
	S8	65536	102169.26	275233.24	641.45
Cuishen-768	S1	32	344.30	927.50	92.94
	S2	128	677.18	1824.24	189.02
	S3	512	1680.21	4526.32	304.72
	S4	1024	3017.47	8128.76	339.36
	S5	4096	11066.42	29811.77	370.13
	S6	8192	21751.52	58596.30	376.62
	S7	16384	43145.93	116230.61	379.73
	S8	65536	171507.38	462022.84	382.12
Cuishen-1024	S1	32	343.84	926.28	93.07
	S2	128	693.63	1868.56	184.54
	S3	512	1694.75	4565.49	302.11
	S4	1024	3025.27	8149.77	338.48
	S5	4096	11021.10	29689.67	371.65
	S6	8192	21665.06	58363.40	378.12
	S7	16384	42954.05	115713.70	381.43
	S8	65536	170652.41	459719.67	384.03

Table 7: x86 AVX-512 C implementation, NGCC S1–S8 performance results.

Version	Level	Message Length (Bytes)	Execution Time (ns)	Clock Cycles (cycles)	Throughput (MB/s)
Cuishen-512	S1	32	173.37	467.04	184.58
	S2	128	341.28	919.37	375.06
	S3	512	865.66	2331.99	591.46
	S4	1024	1556.61	4193.35	657.84
	S5	4096	5703.42	15364.41	718.17
	S6	8192	11225.08	30239.20	729.79
	S7	16384	22275.74	60008.49	735.51
	S8	65536	88607.97	238700.58	739.62
Cuishen-768	S1	32	237.04	638.57	135.00
	S2	128	469.26	1264.13	272.77
	S3	512	1167.12	3144.10	438.69
	S4	1024	2097.07	5649.29	488.30
	S5	4096	7684.23	20700.51	533.04
	S6	8192	15133.09	40766.95	541.33
	S7	16384	30022.36	80877.09	545.73
	S8	65536	119365.47	321558.03	549.04
Cuishen-1024	S1	32	234.56	631.89	136.42
	S2	128	468.90	1263.16	272.98
	S3	512	1165.12	3138.70	439.44
	S4	1024	2093.22	5638.91	489.20
	S5	4096	7669.75	20661.51	534.05
	S6	8192	15094.62	40663.32	542.71
	S7	16384	29952.34	80688.46	547.00
	S8	65536	119088.24	320811.19	550.31

The S1–S8 measurements show the expected transition from fixed-cost behavior to steady-state block processing. For S1 and S2, the measured time includes a large fraction of function-entry, padding, tail handling, and finalization overhead, so the throughput is lower. As the input length increases, this overhead is amortized and the MB/s values approach a stable plateau determined mainly by the Cuishen compression loop. Cuishen-512 has the highest throughput because it uses the smallest parameter set, while Cuishen-768 and Cuishen-1024 show very similar long-message throughput since their optimized software paths have nearly the same steady-state core cost. Across the x86 results, the AVX-512 C implementation improves short-message latency and gives the best long-message throughput for all three x86 parameter sets. The ARMv8 results also scale smoothly with message length, confirming that the implementation benefits from the word-level and vector-lane parallelism available inside each compression block.

Table 8: NGCC resource-consumption results. Static total is `text+data+bss`; peak RSS is the maximum resident set size observed during S1–S8 execution.

Platform	Version	Text (Byte)	Data (Byte)	BSS (Byte)	Static Total (Byte)	Peak RSS (Byte)
ARMv8 SHA3/XAR	Cuishen-512	23963	948	12	24923	1441792
	Cuishen-768	37607	948	12	38567	1572864
	Cuishen-1024	37727	948	12	38687	1572864
x86 AVX2/BMI2	Cuishen-512	17117	864	8	17989	1261568
	Cuishen-768	24030	864	8	24902	1265664
	Cuishen-1024	23910	864	8	24782	1269760
x86 AVX-512	Cuishen-512	18921	848	16	19785	1011712
	Cuishen-768	18485	848	16	19349	1015808
	Cuishen-1024	21459	848	16	22323	1015808

Table 8 shows that all optimized implementations keep a modest static footprint and low runtime memory use. The ARMv8 SHA3/XAR implementation has the largest static code footprint for the larger parameter sets because it keeps both the SHA3/XAR and fallback ARM

paths. On x86, the AVX2/BMI2 implementation occupies more static space for Cuishen-768 and Cuishen-1024 than the AVX-512 implementation, reflecting the cost of the hand-optimized assembly path, while the AVX-512 implementation remains comparatively compact for the larger parameter sets. The peak RSS values remain close across platform families, indicating that the benchmark harness and buffering strategy dominate runtime memory usage rather than the hash state itself. At this current throughput and memory footprint, Cuishen is compatible with real-world software deployment.

6.2 Hardware Implementation

This subsection presents the Verilog hardware simulation implementation results of the Cuishen-512, Cuishen-768 and Cuishen-1024 algorithms. Synthesis was performed in Synopsys Design Compiler L-2016.03-SP1, targeting the TSMC 65 nm process library `tcbn65gplusbwp12tbc_ccs`. The Cuishen-512 implementation was synthesised under a clock frequency of 571.4 MHz. The Cuishen-1024 implementation was synthesised under a clock frequency of 500 MHz. Note that the Cuishen-768 and Cuishen-1024 variants share the same execution process, differing only in their initial vectors and the truncation of the final hash value from 1024 bits to 768 bits. Consequently, they achieve the exact same hardware performance.

The implementation results are shown in Table 9.

Table 9: Hardware implementation performance results

Version	Message Length (Kb)	Clock Cycles (cycles)	Throughput (Mb/s)	Latency (ns)	Synthesized Area (μm^2)	Normalized Area (GE)	Power (mW)
Cuishen-512	1	135	4334	236.25	119285.28	62127.75	10.8214
	8	604	7750	1057			
	256	17220	8699	30135			
	8192	548932	8732	960631			
Cuishen-768/1024	1	135	3793	270	168785.76	87909.25	15.4755
	8	604	6781	1208			
	256	17220	7612	34440			
	8192	548932	7641	1097864			

7 Design Rationale

In this section, we present the main ideas and rationales of the design of Cuishen. The core design idea of Cuishen lies in combining a variant of a double-block length hash mode by Naito [9] with a dedicated ARX block cipher Octarx, which enables high performance and high security.

7.1 Properties of Overall Structure and Components

7.1.1 Overall Structure

The overall structure of Cuishen is inspired by the double-block-length mode proposed by Naito [8], and incorporates a counter in each iteration to improve security against second preimage attacks. Furthermore, Cuishen can absorb one partial message block in the finalization phase thanks to the dedicated block cipher Octarx with a large key size. These properties are described in detail below.

- **Merkle-Damgård iteration with finalization function.** Traditional double-block-length hash constructions like Hirose construction [6], despite possessing optimal collision and preimage resistance, can be differentiated from a random oracle within constant-level number of queries, thus its indistinguishability security is not guaranteed. Cuishen adopts the result by Naito [8] that in a double-block-length mode, provided there is an appropriate finalization function, $O(2^n)$ indistinguishability can be guaranteed.
- **Incorporating a counter in the iteration.** To improve security against second preimage attacks, we incorporate a counter in each iteration of Cuishen. This counter is used to record the cumulative number of processed message bits up to the current iteration. As shown in previous analyses [1, 2], the incorporation of a counter in the iteration can improve security against second preimage attacks.
- **Absorbing message block in the finalization.** Thanks to the large key size of the dedicated block cipher Octarx, Cuishen can absorb one partial message block during the finalization phase. As a result, short messages require only a single compression function call (i.e., two parallel block-cipher calls) to produce the digest. Specifically, this applies to messages of at most 768 bits for Cuishen-512 and at most 512 bits for Cuishen-768 and Cuishen-1024. Consequently, Cuishen achieves high efficiency for short-message hashing.
- **Zero Padding.** Cuishen uses 0* padding instead of 10* padding with length encoding to avoid processing an additional padding block whenever possible. This is achieved through two mechanisms. First, a 128-bit position counter recording the cumulative bit length of the original message is injected in each iteration. Second, the finalization function absorbs the remaining partial message block by encrypting constant plaintext to distinguish the finalization phase from the iterative processing phase.

7.1.2 Algorithm Component: the Octarx Block Cipher

In the design of the underlying block cipher component, Octarx mainly balances security, software implementation efficiency, and hardware implementation cost. It consists of three aspects: the overall encryption structure, the key schedule, and the selection of specific parameters. The details are as follows:

- **ARX structure.** The overarching design philosophy of the Octarx algorithm is to adopt a simple ARX structure, using modular addition as the main nonlinear component, combined with XOR and circular shift operations to achieve rapid mixing of state information. Compared with designs based on S-boxes and complex linear layers, the ARX structure does not rely on lookup operations and has a more regular calculation format, which better suits the word-level calculation characteristics of modern 64-bit processors. Furthermore, to improve implementation efficiency, all modular addition operations in the round function are implemented in parallel. Consequently, Octarx-256 employs two modular additions, while the Octarx-512 variant utilizes four. To further accelerate diffusion, a 64-bit cyclic shift operation is introduced on each of the two inputs of the modular additions. For achieving rapid full diffusion, Octarx adopts a simple compression-expansion mechanism using a few of XOR operations, thereby allowing a lot of input bits to participate in subsequent modular additions. Finally, to resist structural attacks such as slide attacks, a 64-bit round constant is injected into each round of the Octarx cipher.
- **Tweakey-like key schedule.** The key schedule of the Octarx algorithm comprises three primary components, each corresponding to a specific part within the Cuishen hash function: the message processing part (message expansion algorithm), the chaining-value mixing part (chaining-value update algorithm), and the counter update part, which records

the number of processed message bits (counter update algorithm). The design philosophy of the message expansion algorithm is analogous to that of the overall encryption structure. However, because the message block lengths across all three Cuishen versions exceed the block size of the underlying block cipher, and each round function accepts only four 64-bit message words, the message expansion algorithm updates merely four 64-bit words per round. The design of the chaining-value update algorithm is relatively straightforward. Given the inherent difficulty in controlling the chaining-values within the Cuishen algorithm, this component exclusively employs cyclic shift operations to guarantee that the output period is no less than 64. Similarly, the counter update algorithm is designed with simplicity as its primary objective. Its core principle relies on cyclic shift operations to sequentially update the value injected into each round function. This mechanism ensures that the counter value returns to its initial state after exactly 64 updates, thereby preparing it for the counting process in the subsequent compression function.

- **Heuristic cyclic shift parameter selection.** The Octarx cipher incorporates numerous cyclic shift parameters, with the value for each position ranging from 0 to 63. Given that an exhaustive search over all possible parameter combinations is computationally infeasible, we adopted a heuristic approach for parameter selection. For the core encryption process, we randomly sampled over a thousand parameter sets and shortlisted a few candidates based on two primary criteria: the number of rounds required to achieve full diffusion, and the differential probabilities in reduced-round variants. A similar methodology was applied to select candidate cyclic shift parameters for the message expansion algorithm. For the chaining-value update algorithm, the cyclic shift parameters were specifically chosen to maximize the output period. Conversely, for the counter update algorithm, the parameters were selected to ensure an output period of exactly 64. Finally, the ultimate combination of cyclic shift parameters was determined by evaluating the overall related-key differential probabilities of the Octarx cipher in reduced-round settings.

7.2 Parameter Selection

The design of this algorithm fully considers the diverse requirements for security and performance in different application scenarios. Through flexible parameter configurations of the mode and the underlying block cipher Octarx, a graded security design is achieved.

- **Digest length and Block size.** Depending on the different digest length $t \in \{512, 768, 1024\}$, the block size $n \in \{256, 512\}$ of the underlying block cipher Octarx can be selected accordingly, corresponding to an internal state size of $2n \in \{512, 1024\}$ bits in double-block mode. Especially when $n = 512$, Cuishen possesses an internal state of up to 1024 bits, even if the final output is truncated to 768 bits for Cuishen-768. These flexible parameters guarantee that each instance of Cuishen can achieve the corresponding security level.
- **Iteration Rounds r .** Octarx uses $r = 64$ rounds of iteration for both $n = 256$ and $n = 512$ versions, respectively. The basis for this design lies in the diffusion property of the underlying ARX structure: as the block size increases (for example, 512 bits being divided into eight 64-bit words processed in parallel), the amount of data participating in the modular addition, XOR, and shift mixings in a single round is larger. Concurrently, as the key size grows, the number of rounds must be increased to maintain sufficient security margins. Therefore, the Octarx-256 variant requires more rounds against related-key and known-key attacks, because of its key-to-block size ratio of $1408/256 = 5.5$. Conversely, although the Octarx-512 variant has a lower ratio of $1664/512 = 3.25$, its block size is twice that of the Octarx-256 variant. To ensure robust resistance against various attacks, the number of rounds for Octarx-512 is still set to 64.

8 Feature Declarations

In this section, we briefly discuss the features of our hash algorithm, including innovativeness, simplicity, flexibility, and potential performance advantages.

8.1 Innovativeness

- **Counter in the iteration.** A 128-bit position counter is injected into double-block-length hash mode to record the cumulative number of processed message bits. This position-dependent design breaks the position invariance of the iterative structure, which effectively mitigates the threat from expandable message attacks and long-message second-preimage attacks.
- **Message absorption in finalization.** Thanks to the large key size of the dedicated block cipher **Octarx**, the finalization function can absorb the remaining partial message block. Together with the position counter, this finalization mechanism enables efficient 0^* padding as an alternative to the traditional 10^* padding with MD strengthening.
- **Parallel generalized Feistel variant.** The **Octarx** cipher adopts a variant of the generalized Feistel structure. Instead of employing S-boxes and complex linear diffusion matrices, it leverages the synergy of modular addition and cyclic shifts to achieve rapid confusion and diffusion, while simultaneously ensuring the parallel execution of modular addition operations. Compared to traditional generalized Feistel structures, this algorithm incorporates a compression-expansion substructure to accelerate full diffusion. Its overall architecture is both simple and intuitive, delivering excellent hardware and software implementation performance without compromising security.
- **Hash-specific expansion/update algorithms used in **Octarx**.** The key in the **Octarx** cipher corresponds to the message, partial chaining-value, and counter of the **Cuishen** algorithm, resulting in a significantly larger key size. Unlike traditional standalone block ciphers, the **Octarx** is primarily designed to serve hash functions, meaning that only the key positions corresponding to the message and counter are directly controllable. Therefore, instead of directly constructing an iterative key schedule, the **Octarx** adopts the tweakey design idea to process these three key parts separately. Specifically, it emphasizes the key expansion for the message portion while simplifying the key update for the partial chaining-value and counter. This approach not only ensures the rapid integration of each key part into the algorithm but also enhances overall implementation efficiency.

8.2 Simplicity

The underlying core is built entirely on extremely simple ARX (modular Addition, Rotation, XOR) operations. This not only avoids complex algebraic structures but also makes the algorithm naturally immune to Cache-based timing side-channel attacks.

The entire hash algorithm is built solely upon the block cipher **Octarx**. The dual-parallel compression function and the finalization function both reuse the same encryption logic. Furthermore, the two parallel encryption paths share the same expanded key, eliminating redundant message-expansion overhead.

8.3 Flexibility and Extensibility

Cuishen can serve as a flexible multi-functional cryptographic primitive with the following capabilities.

- **eXtendable-Output Function (XOF).** The XOF mode is realized through domain separation. By fixing the finalization key ($H_N \parallel m_{final} \parallel Cnt$) and encrypting successive plaintext constants ($[2j]_n$ and $[2j+1]_n$), Cuishen generates an arbitrary-length pseudorandom output stream. From an implementation perspective, the key schedule is computed only once, and each output block can be generated independently in parallel.
- **Message Authentication Code (MAC).** The indistinguishability of Cuishen suggests that it can serve as a secure replacement for a random oracle in many cryptographic applications. In particular, if Cuishen is indistinguishable from a random oracle, then simple prefix-MAC constructions of the form $MAC_K(M) = \text{Cuishen}(K \parallel M)$ can potentially be used without resorting to nested constructions such as HMAC.
- **Key Derivation Function (KDF):** With its XOF mode, Cuishen naturally serves as a flexible key derivation function (KDF), enabling the derivation of multiple cryptographic keys of arbitrary lengths from a single master secret.
- **Single-block-length mode.** It's worth noting that our current mode is dual-path parallel encryption. We can easily adjust it to construct a single-path encryption mode. This allows our compression function to absorb more message bits each time as the half chaining value is no longer be absorbed as part of the key, thereby increasing throughput.
- **Extensible to Cuishen-256.** Accordingly, if the word size is reduced from 64 bits to 32 bits, the Cuishen-512 variant can be adapted into the Cuishen-256 variant, which produces a 256-bit digest and is tailored for 32-bit software platforms. In this scenario, the cyclic shift parameters within the Cuishen-256 variant should be redefined.

8.4 Implementation Performance Advantages

Cuishen combines modern cryptographic structural innovations with minimalist block cipher design. The two underlying encryption operations inside the compression function can be implemented independently and share the same key schedule. The Superscalar Pipeline of modern CPUs can easily dispatch these parallel ARX flows to different execution ports, maximizing Instructions Per Cycle (IPC). The layout is exceptionally well-suited for Single Instruction, Multiple Data (SIMD) vectorization (such as AVX2/AVX-512 and NEON).

References

- [1] E. Biham and O. Dunkelman. A framework for iterative hash functions - HAIFA. *IACR Cryptol. ePrint Arch.*, 2007:278, 2007.
- [2] C. Bouillaguet and B. Vayssière. Provable second preimage resistance revisited. In *Selected Areas in Cryptography - SAC 2013 - 20th International Conference, Burnaby, BC, Canada, August 14-16, 2013, Revised Selected Papers*, pages 513–532, 2013.
- [3] G. Brassard, P. Høyer, and A. Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN '98: Theoretical Informatics, 3rd Latin American Symposium, Campinas, Brazil, April 20-24, 1998, Proceedings*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.
- [4] A. Chailloux, M. Naya-Plasencia, and A. Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 211–240. Springer, 2017.
- [5] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.
- [6] S. Hirose. Some plausible constructions of double-block-length hash functions. In *Fast Software Encryption, 13th International Workshop, FSE 2006, Graz, Austria, March 15-17, 2006, Revised Selected Papers*, pages 210–225, 2006.
- [7] J. Kelsey and B. Schneier. Second preimages on n -bit hash functions for much less than 2^n work. In *Fast Software Encryption, 12th International Workshop, FSE 2005, Paris, France, February 21-23, 2005, Revised Selected Papers*, volume 3557 of *Lecture Notes in Computer Science*, pages 474–490. Springer, 2005.
- [8] Y. Naito. Blockcipher-based double-length hash functions for pseudorandom oracles. In A. Miri and S. Vaudenay, editors, *Selected Areas in Cryptography - 18th International Workshop, SAC 2011, Toronto, ON, Canada, August 11-12, 2011, Revised Selected Papers*, volume 7118 of *Lecture Notes in Computer Science*, pages 338–355. Springer, 2011.
- [9] Y. Naito. Indifferentiability of double-block-length hash function without feed-forward operations. In *Information Security and Privacy - 22nd Australasian Conference, ACISP 2017, Auckland, New Zealand, July 3-5, 2017, Proceedings, Part II*, pages 38–57, 2017.